EGE UNIVERSITY

FACULTY OF ENGINEERING

COMPUTER ENGINEERING DEPARTMENT

PROGRAMMING LANGUAGES

Lexical Analyzer For the Plus++ Language

2024–2025 SPRING SEMESTER

PROJECT-1 REPORT

PREPARED BY

YİĞİT ÇAKAR 05220000329

GÖRKEM EGE AKINCI 05220000315

Contents

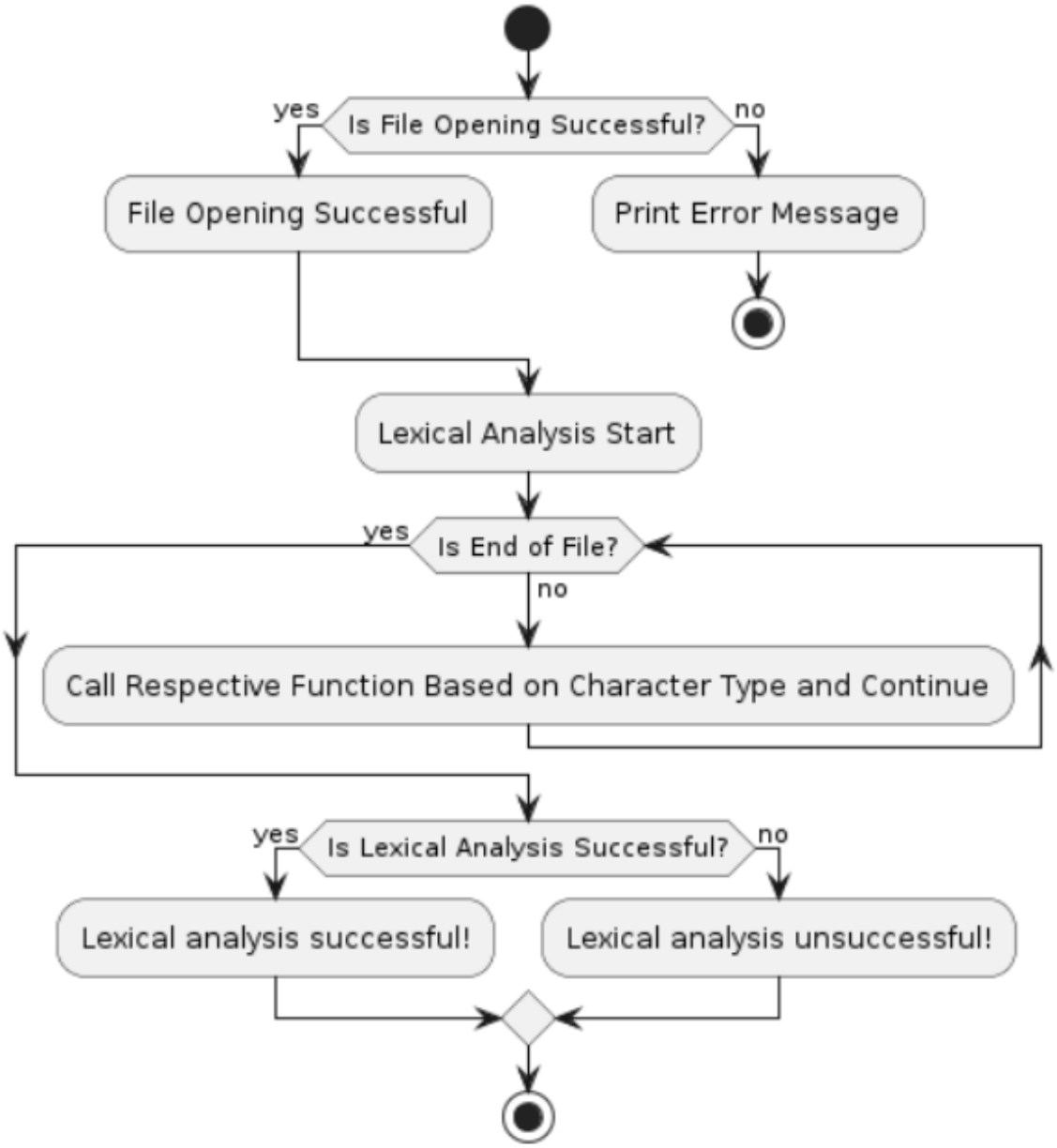
Introduction and Flowchart 2

Requirements 4

1. Token Types..... 4
2. Token Structure..... 5
3. Reading a Character..... 5
4. Skipping Blanks and Comments..... 6
5. Keywords..... 6
6. Reading a Token..... 7
7. Writing a Token.....11
Main Method 12
Examples and outputs 13

Introduction and Flowchart

The Plus++ programming language is a small and simple scripting language designed to help students understand how programming languages work behind the scenes—especially how a lexical analyzer processes code. It uses only one data type, the number, which represents whole numbers. In Plus++, you can declare variables, assign them values, increase or decrease their values, and display results using the write command. Strings can also be printed, but only as constants. The language includes basic control flow with loops and blocks, making it ideal for beginners to grasp core concepts. In this report, we'll walk through how Plus++ works, what it supports, how it handles errors, and how we built a lexer to break down its code step by step.


Is File Opening Successful?
yes
no
File Opening Successful
Print Error Message
Lexical Analysis Start
Is End of File?
yes
no
Call Respective Function Based on Character Type and Continue
Is Lexical Analysis Successful?
yes
no
Lexical analysis successful!
Lexical analysis unsuccessful!

Requirements

1)Token types

```
// Token Tipleri
typedef enum {
    T_KEYWORD,          // Anahtar kelime (number, write vs.)
    T_IDENTIFIER,       // Değişken ismi
    T_INT_CONSTANT,     // Tam sayı sabiti
    T_STRING_CONSTANT,  // Çift tırnaklı sabit
    T_OPERATOR,         // :=, +=, -= gibi işleçler
    T_OPEN_BLOCK,       // { bloğu başlatır
    T_CLOSE_BLOCK,      // } bloğu kapatır
    T_END_OF_LINE       // ; satır sonu
} TokenType;
```

The code block defines an enum named TokenType, which lists all possible types of **tokens** that the lexical analyzer can recognize in the Plus++ language. Tokens are the smallest units of meaning in the source code, such as keywords, identifiers, numbers, or special symbols.

This enumeration TokenType categorizes each kind of lexical token that may appear in a Plus++ script. These include:

- **Keywords** (like number, write, repeat, etc.) that have predefined meanings in the language.
- **Identifiers**, which are user-defined variable names.
- **Integer constants**, representing whole number values.
- **String constants**, which are sequences of characters enclosed in double quotes.
- **Operators** such as :=, +=, and -=, used for assignment or arithmetic operations.
- **Braces** ({, }), used to define code blocks.
- **Semicolons** (;), used to mark the end of a statement.

```
// Token adlarını string olarak tanımlayan dizi (çıkışta kullanılır)
static const char *TOKEN_NAMES[] = {
    "Keyword", "Identifier", "IntConstant", "StringConstant",
    "Operator", "OpenBlock", "CloseBlock", "EndOfLine"
};
```

This code block defines a **string array** named TOKEN_NAMES[], which is used to map each TokenType enum value to a **human-readable string**.

2)Token Structure

```c
// Token yapısı: her token'ın türü, içeriği (lexeme), bulunduğu satır ve sütun
typedef struct {
    TokenType type;
    char lexeme[MAX_LEXEME];
    int line, col;
} Token;
```

The Token struct represents each unit identified during lexical analysis. It stores the token's type, its text (lexeme), and its position in the source file (line and column). This helps in both analysis and precise error reporting.

3)Reading a character

```c
// Bir karakter oku ve konum sayacını güncelle
static int nextc(FILE *in) {
    int c = fgetc(in);
    if (c == '\n') { g_line++; g_col = 1; }
    else if (c != EOF) { g_col++; }
    return c;
}
```

This function, nextc, reads the next character from the input file and updates the global line (g_line) and column (g_col) counters accordingly. If it encounters a newline character (\n), it increments the line counter and resets the column counter to 1. Otherwise, it increments the column counter. This ensures accurate tracking of character positions for error reporting.

```c
// Bir karakteri sadece kontrol et (geri ver)
static int peekc(FILE *in) {
    int c = fgetc(in);
    if (c != EOF) ungetc(c, in);
    return c;
}
```

The peekc function reads the next character from the input file without consuming it. It uses fgetc to get the character, and if it's not end-of-file (EOF), it immediately pushes it back into the input stream with ungetc. This allows the lexer to look ahead at the next character without advancing the file pointer.

4)Skipping blanks and comments

```c
// Boşlukları ve *yorumları* atla
static void skip_ws_and_comments(FILE *in) {
    int c;
    while ((c = peekc(in)) != EOF) {
        if (isspace(c)) { nextc(in); continue; } // Boşluksa atla
        if (c == '*') {                          // Yorum başı
            int sl = g_line, sc = g_col;         // Hata durumunda yer bilgisi
            nextc(in);                           // İlk * karakterini al
            while ((c = nextc(in)) != EOF && c != '*') ; // Son *'i arıyoruz
            if (c == EOF) {                      // Eğer * kapanmadıysa
                fprintf(stderr, "Error [%d:%d] Unterminated comment\n", sl, sc);
                exit(EXIT_FAILURE);
            }
            nextc(in); // Kapatıcı '*' karakterini tüket
            continue;
        }
        break;
    }
}
```

The skip_ws_and_comments function is responsible for skipping over whitespace characters and comment blocks in the input stream. It repeatedly checks the next character using peekc and consumes it if it is a whitespace (such as spaces, tabs, or newlines). If a comment is detected (indicated by an asterisk *), the function continues reading characters until it finds a closing asterisk, effectively skipping the entire comment. If the end of the file is reached before the closing asterisk is found, it reports an "Unterminated comment" error along with the exact line and column location, then exits.

5)Keywords

```c
// Plus++ dilinde geçerli olan anahtar kelimeler
static const char *KEYWORDS[] = {
    "number", "repeat", "times", "write", "and", "newline", NULL
};
```

This block defines a **list of reserved keywords** in the Plus++ language.

```c
// Belirtilen kelime anahtar kelime mi kontrol et
static int is_keyword(const char *w) {
    for (int i = 0; KEYWORDS[i]; i++)
        if (strcmp(w, KEYWORDS[i]) == 0) return 1;
    return 0;
}
```

The is_keyword function is responsible for checking whether a given string is a reserved keyword in the Plus++ language. It iterates through the predefined KEYWORDS array, comparing the input string to each keyword using the strcmp function. If a match is found, it returns 1 (true), indicating that the word is a keyword. Otherwise, it returns 0 (false).

6)Reading a Token

```c
// Bir sonraki token'ı oku ve yapı olarak döndür
static int next_token(FILE *in, Token *tk) {
    skip_ws_and_comments(in);
    int c = peekc(in);
    if (c == EOF) return 0; // Dosya sonu

    tk->line = g_line;      // Token'ın başladığı yer
    tk->col  = g_col;
    c = nextc(in);          // Karakteri al
```

The next_token function is the core of the lexical analyzer. It reads and returns the next token from the input stream. First, it skips over any whitespace or comments using the skip_ws_and_comments function. Then, it peeks at the next character to check if the end of the file has been reached—if so, it returns 0 to signal no more tokens. Otherwise, it stores the current line and column positions to record where the token begins. Finally, it consumes the character with nextc(in) to begin token classification. This function sets the stage for determining the type and content of the token to be returned.

- <u>Blocks, end of line characters</u>

```c
// Blok, satır sonu karakterleri
if (c == '{') { tk->type = T_OPEN_BLOCK;  strcpy(tk->lexeme, "{"); return 1; }
if (c == '}') { tk->type = T_CLOSE_BLOCK; strcpy(tk->lexeme, "}"); return 1; }
if (c == ';') { tk->type = T_END_OF_LINE; strcpy(tk->lexeme, ";"); return 1; }
```

This part of the next_token function handles block delimiters and line terminators. If the current character is a left curly brace {, it classifies the token as T_OPEN_BLOCK, copies the brace into the token's lexeme, and returns success. Similarly, if it encounters a right curly brace }, it creates a T_CLOSE_BLOCK token. For a semicolon ;, it generates a T_END_OF_LINE token. These checks ensure that block structures and line endings in the Plus++ language are recognized as distinct tokens during lexical analysis.

- <u>Operators</u>

```c
// Operatörler (:=, +=, -=)
if (c == ':' || c == '+' || c == '-') {
    int next = nextc(in);
    if (next == '=') {
        tk->type = T_OPERATOR;
        tk->lexeme[0] = (char)c;
        tk->lexeme[1] = '=';
        tk->lexeme[2] = '\0';
        return 1;
    }
    ungetc(next, in);

    if (c == '-' && isdigit(peekc(in))) { // Negatif sayı kontrolü
        int len = 0;
        char buf[MAX_LEXEME];
        buf[len++] = '-';
        c = nextc(in);
        while (isdigit(c) && len < MAX_NUMBER_LENGTH + 1) {
            buf[len++] = (char)c;
            c = nextc(in);
        }
        ungetc(c, in);
        buf[len] = '\0';

        if (len - 1 > MAX_NUMBER_LENGTH) { // Sayı uzunluğu kontrolü
            fprintf(stderr, "Error [%d:%d] Number too long (> %d digits): %s\n", tk->line, tk->col, MAX_NUMBER_LENGTH, buf);
            exit(EXIT_FAILURE);
        }

        tk->type = T_INT_CONSTANT;
        strcpy(tk->lexeme, buf);
        return 1;
    } else {
        fprintf(stderr, "Error [%d:%d] Unknown character\n", tk->line, tk->col);
        exit(EXIT_FAILURE);
```

This section of the code is responsible for identifying operator tokens (:=, +=, -=) and recognizing negative integers. When a colon, plus, or minus sign is encountered, the next character is checked to see if it forms a valid operator with an equals sign. If a minus sign is followed by a digit, the code interprets it as a negative number. The negative number is then parsed, ensuring that it does not exceed the maximum allowed length of 100 digits, as required by the Plus++ language specification. If it exceeds the limit or any invalid sequence is detected, an error is reported and the program terminates.

- Integer constants (numbers)

```c
// Tam sayı sabiti
if (isdigit(c)) {
    int len = 0;
    char buf[MAX_LEXEME];
    do {
        buf[len++] = (char)c;
        c = nextc(in);
    } while (isdigit(c) && len < MAX_NUMBER_LENGTH);

    ungetc(c, in);
    buf[len] = '\0';

    if (len > MAX_NUMBER_LENGTH) {
        fprintf(stderr, "Error [%d:%d] Number too long (> %d digits): %s\n", tk->line, tk->col, MAX_NUMBER_LENGTH, buf);
        exit(EXIT_FAILURE);
    }

    tk->type = T_INT_CONSTANT;
    strcpy(tk->lexeme, buf);
    return 1;
}
```

This part of the lexer is responsible for processing integer constants. When a digit is encountered, the lexer collects all subsequent digits to form a complete number token. The number is stored in a buffer with a maximum length defined by the Plus++ specification (100 digits). If the number exceeds this limit, an error message is displayed, and the program terminates. This ensures that all numeric literals in the source file conform to the language's data type constraints.

- String constants

```c
// String sabiti
if (c == '\"') {
    int len = 0;
    char buf[MAX_LEXEME];
    buf[len++] = '\"';
    while ((c = nextc(in)) != EOF && c != '\"' && len < MAX_LEXEME - 2)
        buf[len++] = (char)c;
    if (c == EOF) {
        fprintf(stderr, "Error [%d:%d] Unterminated string\n", tk->line, tk->col);
        exit(EXIT_FAILURE);
    }
    buf[len++] = '\"';
    buf[len] = '\0';
    tk->type = T_STRING_CONSTANT;
    strcpy(tk->lexeme, buf);
    return 1;
}
```

This code block is responsible for recognizing and processing string constants in the source code. It begins by checking if the current character is a quotation mark ("), indicating the start of a string. It then reads and stores all subsequent characters until it encounters another quotation mark or reaches the end of the file. If the string is not properly closed with a quotation mark before the file ends, it reports an "Unterminated string" error along with the line and column number where the string began. Once the string is successfully read, it is stored in the token's lexeme field, its type is set to T_STRING_CONSTANT, and the function returns success.

- Identifiers or keywords

```c
// Identifier veya keyword
if (isalpha(c)) {
    int len = 0;
    char buf[MAX_LEXEME];
    buf[len++] = (char)c;
    c = nextc(in);
    while ((isalnum(c) || c == '_') && len < MAX_LEXEME - 1) {
        buf[len++] = (char)c;
        c = nextc(in);
    }
    ungetc(c, in);
    buf[len] = '\0';

    if (!is_keyword(buf) && strlen(buf) > MAX_IDENTIFIER_LENGTH) {
        fprintf(stderr, "Error [%d:%d] Identifier too long (> %d characters): %s\n", tk->line, tk->col, MAX_IDENTIFIER_LENGTH, buf);
        exit(EXIT_FAILURE);
    }

    if (!isalpha(buf[0])) { // İlk karakter harf olmalı
        fprintf(stderr, "Error [%d:%d] Identifier must begin with a letter: %s\n", tk->line, tk->col, buf);
        exit(EXIT_FAILURE);
    }

    if (is_keyword(buf))
        tk->type = T_KEYWORD;
    else
        tk->type = T_IDENTIFIER;

    strcpy(tk->lexeme, buf);
    return 1;
}
```

Identifiers in Plus++ are used to name variables and must comply with specific lexical rules. The scanner identifies a potential identifier when a character starts with an alphabetical letter. It then reads the subsequent characters as long as they are alphanumeric or underscores, up to a maximum length. If the identifier exceeds 20 characters or starts with a non-letter character, an error is raised. Additionally, the scanner checks whether the parsed word matches a keyword. If so, it is categorized as a keyword; otherwise, it is treated as a valid identifier. This ensures both syntactic correctness and clear distinction between reserved words and user-defined variables.

- Undefined characters

```c
// Tanımsız karakter
fprintf(stderr, "Error [%d:%d] Unknown character '%c'\n", tk->line, tk->col, c);
exit(EXIT_FAILURE);
```

This code block handles unrecognized or invalid characters encountered during lexical analysis. If none of the defined token patterns match the current character, the analyzer prints an error message to the standard error stream (stderr), indicating the exact line and column where the unknown character was found. The character itself is also included in the message for clarity. After displaying the error, the program immediately terminates using exit(EXIT_FAILURE);, ensuring that the analyzer does not continue processing invalid input. This helps maintain strict syntax validation in the Plus++ language.

7)Writing a Token

```c
// Token'ı dosyaya yaz
static void write_token(FILE *out, const Token *tk) {
    if (tk->type == T_END_OF_LINE || tk->type == T_OPEN_BLOCK || tk->type == T_CLOSE_BLOCK)
        fprintf(out, "%s\n", TOKEN_NAMES[tk->type]);
    else
        fprintf(out, "%s(%s)\n", TOKEN_NAMES[tk->type], tk->lexeme);
}
```

This function, write_token, outputs the recognized token to a file. It takes a file pointer out and a pointer to a Token structure tk. If the token is a simple symbol like EndOfLine, OpenBlock, or CloseBlock, it writes only the token type's name (like EndOfLine) on a new line. For other tokens that include an associated lexeme (like keywords, identifiers, operators, etc.), it writes both the token type and its lexeme in the format Type(Lexeme) (e.g., Identifier(myVar)). This output format is used to generate a readable .lx file from the lexical analysis process.

Main Method

```c
// Ana fonksiyon
int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: la <scriptName>\n");
        return EXIT_FAILURE;
    }

    // Girdi ve çıktı dosya adları oluşturulur
    char in_name[256], out_name[256];
    snprintf(in_name, 256, "%s.plus", argv[1]);
    snprintf(out_name, 256, "%s.lx", argv[1]);

    FILE *in = fopen(in_name, "r");
    if (!in) { perror(in_name); return EXIT_FAILURE; }

    FILE *out = fopen(out_name, "w");
    if (!out) { perror(out_name); return EXIT_FAILURE; }

    // Tokenları sırayla oku ve yaz
    Token tk;
    while (next_token(in, &tk)) {
        write_token(out, &tk);
    }

    // EOF sonrası fazladan karakter kontrolü (yeni eklendi)
    int extra;
    while ((extra = fgetc(in)) != EOF) {
        if (!isspace(extra)) {
            fprintf(stderr, "Warning: Extra character after EOF at [%d:%d]: '%c'\n", g_line, g_col, extra);
            break;
        }
    }

    fclose(in);
    fclose(out);

    // Başarı lexical analysis mesajı
    printf("Lexical analysis complete. Output -> %s\n", out_name);
    return EXIT_SUCCESS;
}
```

The main function handles the overall execution flow of the lexical analyzer. It first checks for the correct usage by validating the input arguments. It then constructs the names of the input (.plus) and output (.lx) files and attempts to open them. The function reads tokens one by one using next_token and writes them to the output file. After all tokens have been processed, it performs an additional check to detect any unexpected characters after the logical end of the file (EOF). If any non-whitespace characters are found beyond EOF, a warning is issued. This helps catch accidental or malformed trailing input. Finally, the function closes the files and prints a success message.

Examples and outputs

- Example given in the project document

myscript.plus

```
number size;
number sum;
size:=5;
repeat size times     *ignore me, I am a comment*
{ write size and newline;  *print to screen*
  sum+=size;
}
write newline and "Sum:" and sum;
```

myscript.lx

```
Keyword(number)
Identifier(size)
EndOfLine
Keyword(number)
Identifier(sum)
EndOfLine
Identifier(size)
Operator(:=)
IntConstant(5)
EndOfLine
Keyword(repeat)
Identifier(size)
Keyword(times)
OpenBlock
Keyword(write)
Identifier(size)
Keyword(and)
Keyword(newline)
EndOfLine
Identifier(sum)
Operator(+=)
Identifier(size)
EndOfLine
CloseBlock
Keyword(write)
Keyword(newline)
Keyword(and)
StringConstant("Sum:")
Keyword(and)
Identifier(sum)
EndOfLine
```

- A comment that starts but does not terminate before the end of file

```
bad_comment.plus
1   number x;
2
3   * yorum kapatılmıyor
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\plson>  & 'c:\Users\Görkem\.vscode\extensions\ms
hby' '--stderr=Microsoft-MIEngine-Error-ixjfthv0.nu1'
Error [3:1] Unterminated comment
PS C:\plson>
```

The error message points exact error type and error location ([3:1] means 3rd row,1st column)

- A string constant that starts but does not terminate before the end of file

```
bad_string.plus
1   write "Hello;
2
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\plson>  & 'c:\Users\Görkem\.vscode\extensions\ms
gdq' '--stderr=Microsoft-MIEngine-Error-3klnhtbs.gfw'
Error [1:8] Unterminated string
PS C:\plson>
```

The error message points exact error type and error location ([1:8] means 1st row, 8th column)

- An unrecognized character is detected in the code

```
bad_char.plus
1   number π;   * (pi sembolü) *
2
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\plson>  & 'c:\Users\Görkem\.vscode\extensions\m
3ym' '--stderr=Microsoft-MIEngine-Error-4pl1r3tv.a23'
Error [1:9] Unknown character 'π'
PS C:\plson>
```

The error message points exact error type and error location ([1:9] means 1st row, 9th column)